Coursework Portfolio 1 — Week 5 Lab Report

Web Application Security Lab: WPScan Penetration Testing

Student ID: SHE23601567

Module Code: CMP-X305-0

Submission Deadline: 31 March 2026

University of Roehampton London

Table of Contents

Web Application Security Lab: WPScan Penetration Testing 1
Table of Figures 4
Introduction 5
Part 1: Lab Environment Setup 6
Step 1: Lab Environment Setup 6
Step 2: Network Verification and IP Configuration 8
Step 3: Connectivity Testing and Web Access Verification 9
Step 4: Adding a Vulnerable Theme and Plugin 11
Part 2: WPScan Lab Report 14
Step 1: Vulnerability Database Update 14
Step 2 - Comprehensive WordPress Scan 15
Step 3: Brute Forcing Login Passwords 20
Step 4: Scanning with API Key for Detailed Vulnerabilities 21
Step 5: Excluding Specific Scans and Setting a Delay 21
Conclusion 23
Reflection 25
Use of Artificial Intelligence 26

Table of Figures

Figure 1: Host-Only network configuration 6
Figure 2: Adapter 2 Kali = NAT 7
Figure 3: Adapter 1 Kali = Host-Only 7
Figure 4: Adapter 1 Wordpress = Host-Only 7
Figure 5: Confirm updated IP address in Wordpress 8
Figure 6: Identify Kali's IP address 9
Figure 7: Verify network connectivity between kali and wordpress 9
Figure 8: Verify internet connectivity 10
Figure 9: Open web browser to confirm connectivity 10
Figure 10: Create wordpress user account with WordPress user account with administrative privileges 11
Figure 11: Insert vulnerable plugin (WP File Manager) 11
Figure 12: All themes incl. vulnerable theme "Twenty Twelve" 12
Figure 13: Updating the WPScan vulnerability database 14
Figure 14: WPScan showing the target URL and scan start information 15
Figure 15: Target WordPress version identified by WPScan 16
Figure 16: Confirmation of XML-RPC service enabled on the target system 17
Figure 17: WPScan confirms the presence of the manually installed WP File Manager plugin. 18
Figure 18: Aksimet plugin identified 19
Figure 19: Verification of database access 19
Figure 20: Elapsed Time and Requests 20
Figure 21:: WPScan command used to enumerate WordPress usernames. 20
Figure 22: WPScan enumeration of WordPress usernames revealing valid accounts. 20
Figure 23: WPScan command configured for username and password enumeration. 21
Figure 24: WPScan identifying valid credentials 21
Figure 25: Targeted WPScan for vulnerable plugins & themes (low-impact scan). 21
Figure 26: Scan for vulnerable plugins and themes complete 22

Introduction

The aim of this lab was to simulate a real-world penetration testing environment in order to identify and analyse security vulnerabilities in a WordPress application using WPScan.

Part 1: Lab Environment Setup

Step 1: Lab Environment Setup

This step involves setting up a controlled penetration testing environment using two virtual machines: a Kali Linux machine (attacker) and a WordPress machine (target). The purpose of this setup is to simulate a real-world scenario in which a security analyst performs vulnerability assessments on a web application.

The virtual machines were configured using a combination of NAT and Host-Only networking. Host-Only networking allows isolated communication between the VMs, while NAT provides internet access for updates and tool installation. This combination ensures both functionality and security

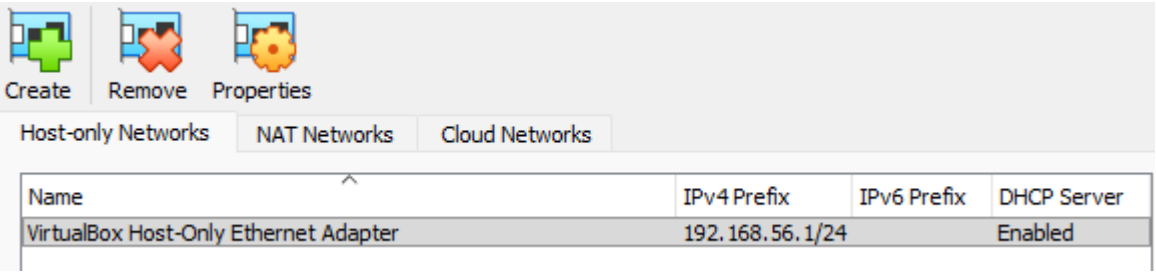

Figure 1: Host-Only network configuration

For Kali:

- Adapter 1: Host only network
    - Allows direct communication between Kali and WordPress
    - Enables penetration testing tools to interact with target system safely
- Adapter 2: Nat
    - Provides internet access to the Kali VM

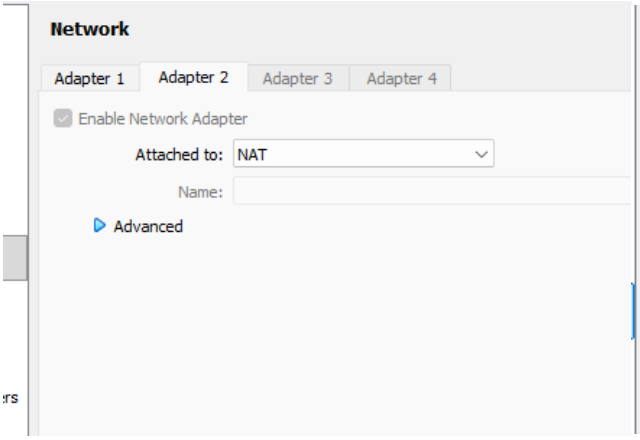

Figure 2: Adapter 2 Kali = NAT

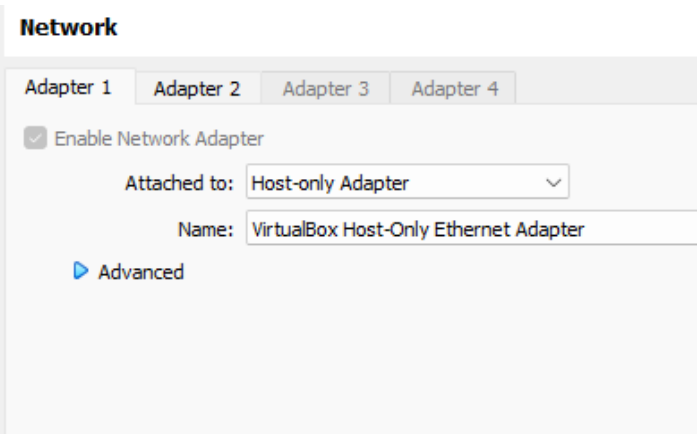

Figure 3: Adapter 1 Kali = Host-Only

For wordpress:

- Adapter 1: Host-only adapter
    - Allows communication with Kali Vm
    - Remains isolated from the internet to create a controlled testing environment

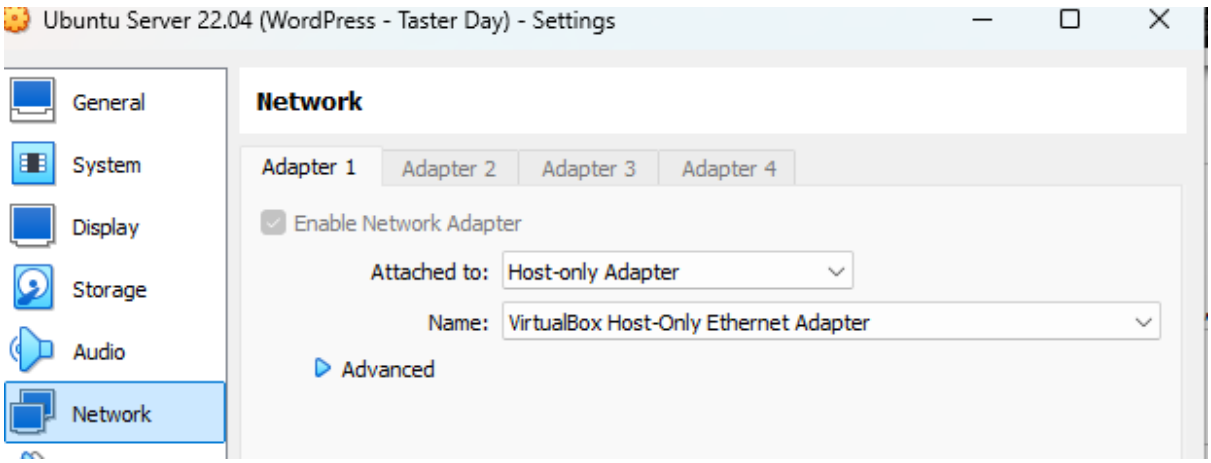

Figure 4: Adapter 1 Wordpress = Host-Only

Step 2: Network Verification and IP Configuration

After configuring the network settings, both virtual machines were started. The ip a command was executed on each machine to display network configuration details, including assigned IP addresses.

On the WordPress virtual machine, the command sudo dhclient was used to request or renew an IP address from the DHCP server. This ensures that the system is correctly assigned an address within the network.

The ip a command was then run again to confirm the updated IP address. The WordPress machine was assigned the IP address 192.168.56.101, which is used for communication within the Host-Only network.

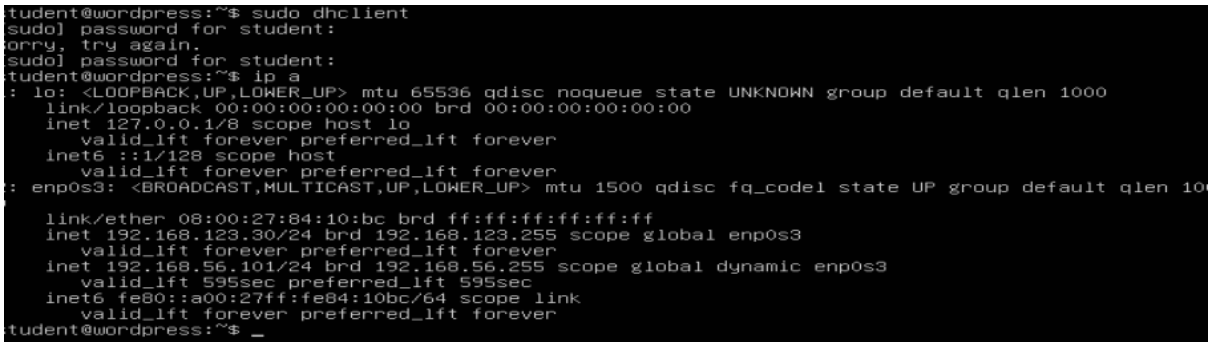

Figure 5: Confirm updated IP address in Wordpress

Step 3: Connectivity Testing and Web Access Verification

The IP address of the Kali Linux machine was identified using the ip a command, which returned 192.168.123.100.

```
Session  Actions  Edit  View  Help
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:63:b0:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.123.100/24 brd 192.168.123.255 scope global noprefixroute et
h0
       valid_lft forever preferred_lft forever
    inet6 fe80::96da:1812:c12f:e0ed/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:9f:37:db brd ff:ff:ff:ff:ff:ff
    inet 192.168.123.100/24 brd 192.168.123.255 scope global noprefixroute et
h1
       valid_lft forever preferred_lft forever
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute eth1
       valid_lft 85317sec preferred_lft 85317sec
    inet6 fe80::fcc0:151f:7ab3:cae8/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

┌──(kali㉿kali)-[~]
└─$
```

Figure 6: Identify Kali's IP address

To verify network connectivity between the two virtual machines, a ping test was performed from Kali Linux to the WordPress machine using: ping 192.168.56.101

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=63 time=1.39 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=63 time=0.847 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=63 time=1.78 ms
^C
--- 192.168.56.101 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 0.847/1.336/1.778/0.381 ms

┌──(kali㉿kali)-[~]
└─$
```

Figure 7: Verify network connectivity between kali and wordpress

Successful responses confirmed that communication between the two systems was functioning correctly.

An additional ping test to an external site (e.g., google.com) was performed to verify internet connectivity via the NAT adapter.

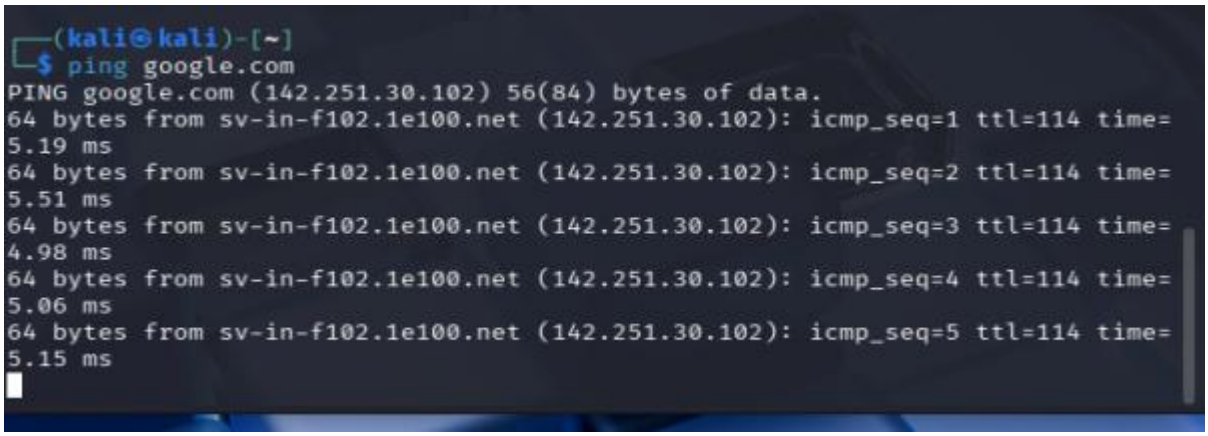

Figure 8: Verify internet connectivity

Following this, a web browser (Firefox) was opened on the Kali Linux machine, and the WordPress IP address was entered.

a. The successful loading of the WordPress page confirmed that the web server was operational and accessible over the network.
b. The presence of the default “Hello World” page indicates that the WordPress installation is functioning correctly and is ready for further security testing.

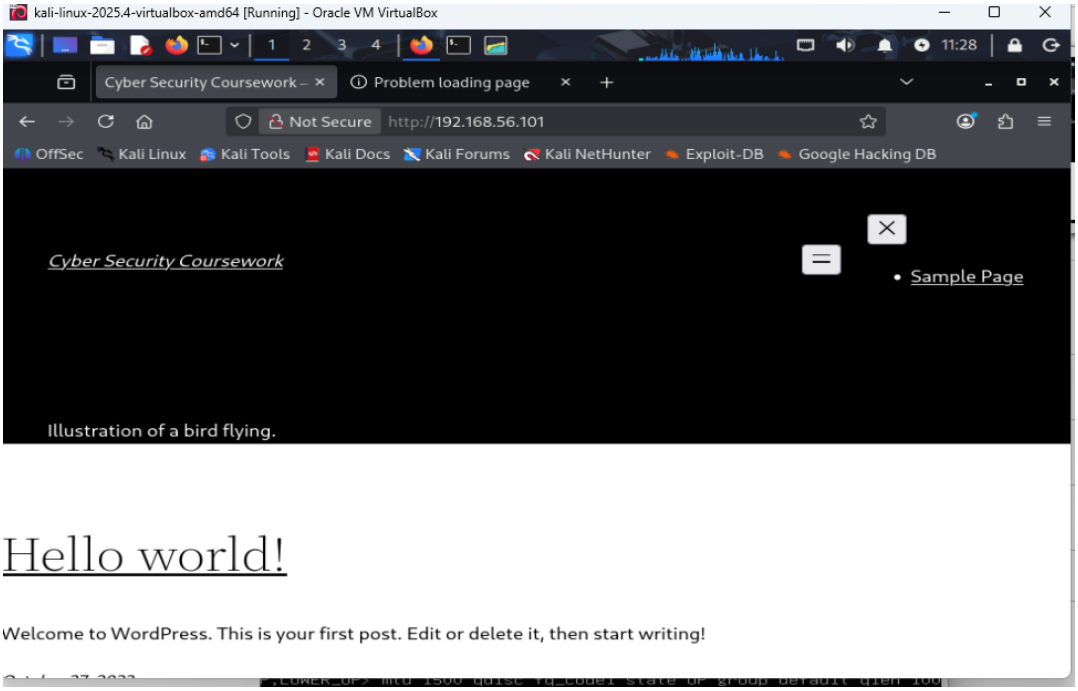

Figure 9: Open web browser to confirm connectivity

To simulate real-world security weaknesses, a new WordPress user account with administrative privileges was created using a weak password. This reflects common security misconfigurations and allows for testing of brute-force attacks.

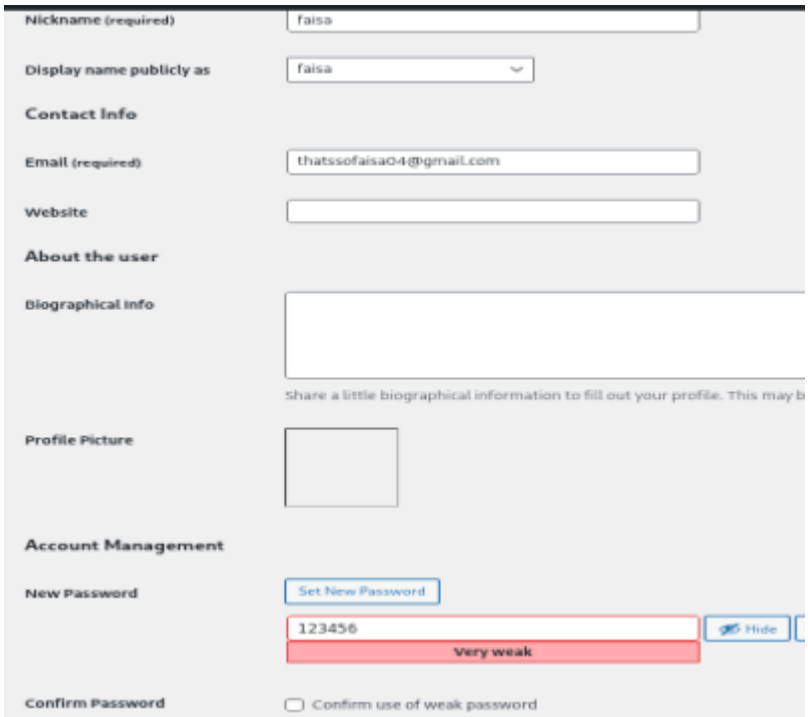

Figure 10: Create wordpress user account with WordPress user account with administrative privileges

Step 4: Adding a Vulnerable Theme and Plugin

To simulate a realistic vulnerable environment, both a vulnerable theme and plugin were introduced into the WordPress installation.

Firstly, a vulnerable plugin (WP File Manager) was manually installed. This is to increase the attack surface.

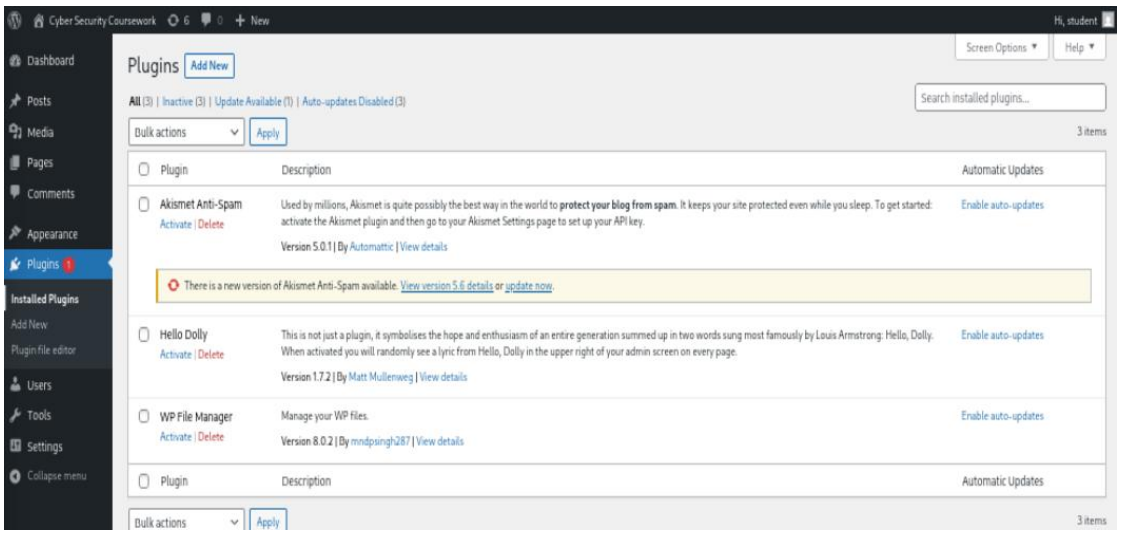

Figure 11: Insert vulnerable plugin (WP File Manager)

A vulnerable theme (Twenty Twelve) was also activated to introduce additional potential security weaknesses within the application. This theme is known to contain outdated components, making it suitable for security testing.

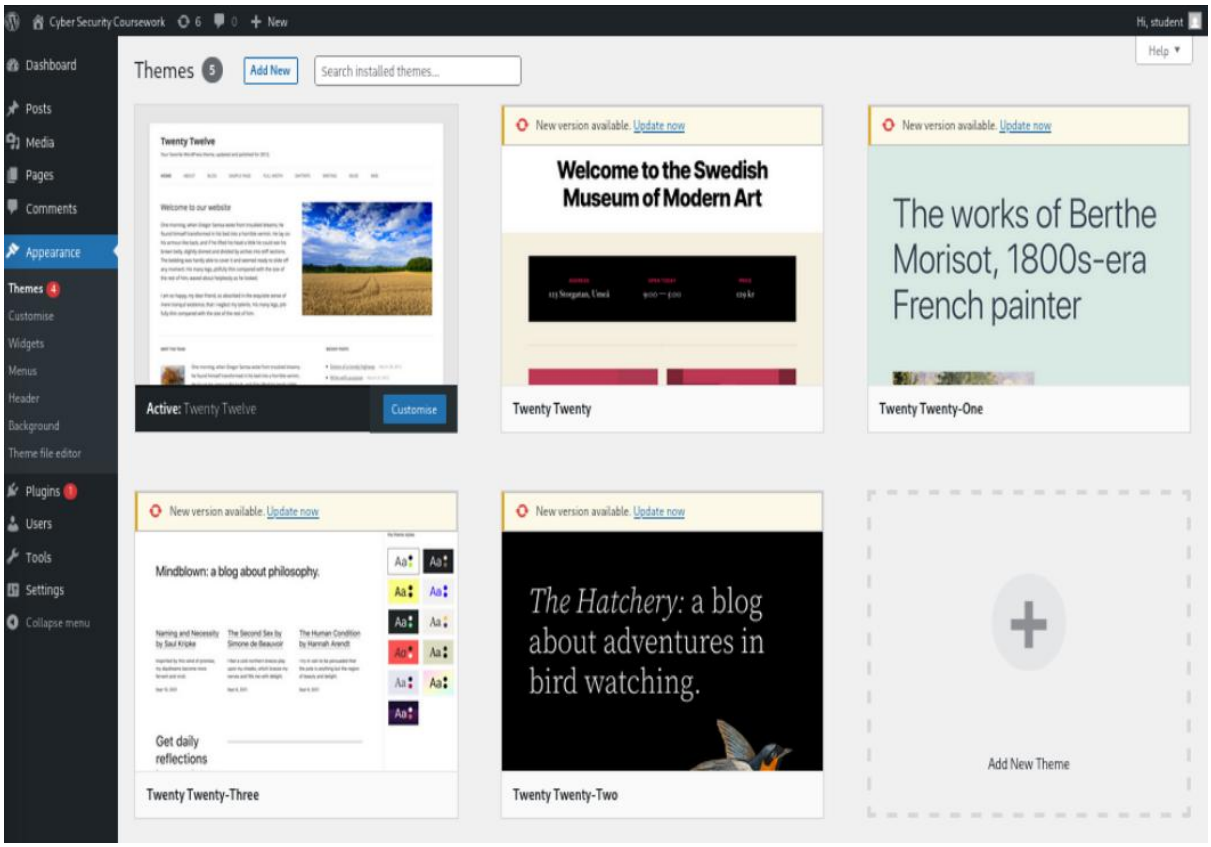

Figure 12: All themes incl. vulnerable theme "Twenty Twelve"

Initially, the WordPress virtual machine did not have internet access, so a NAT network adapter was temporarily added to enable downloading external resources.

The plugin was installed manually via the command line by navigating to the WordPress plugins directory:

```
cd /var/www/wordpress/wp-content/plugins
```

The plugin package was then downloaded using:

```
sudo wget https://downloads.wordpress.org/plugin/wp-file-manager.zip
```

After downloading, the archive was extracted:

```
sudo unzip wp-file-manager.zip
```

This created the plugin directory containing all necessary files. Appropriate permissions were then assigned to ensure the web server (Apache running as www-data) could access the plugin:

```
sudo chown -R www-data:www-data wp-file-manager
```

Finally, the Apache service was restarted to apply the changes:

```
sudo systemctl restart apache2
```

The plugin was then activated through the WordPress admin interface.

This step demonstrates how vulnerable components can be introduced into a web application. In real-world scenarios, outdated or insecure plugins and themes are a common source of vulnerabilities and significantly increase the attack surface.

Part 2: WPScan Lab Report

Step 1: Vulnerability Database Update

Before conducting the security assessment, the WPScan vulnerability database was updated using the command:

Figure 13: Updating the WPScan vulnerability database

Updating the database ensures that WPScan uses the most recent vulnerability signatures and CVE references during the scan. This step is critical because outdated vulnerability data could result in missing newly disclosed security issues. The successful update confirms that the API token was valid and that the scanner had access to the latest vulnerability intelligence from the WPScan database.

Step 2 - Comprehensive WordPress Scan

A full scan was performed against the target WordPress instance to enumerate plugins, themes, and identify the installed WordPress core version. The scan included vulnerability checks against the WPScan database using the API token.

This phase aimed to identify:

- The installed WordPress version
- Known vulnerabilities affecting that version
- Installed plugins and themes
- Vulnerabilities associated with plugins
- Misconfigurations increasing attack surface

Figure 14: WPScan showing the target URL and scan start information

2.1 WordPress Version + Identified Vulnerabilities

```
[+] WordPress version 6.1.1 identified (Insecure, released on 2022-11-15).
 | Found By: Rss Generator (Passive Detection)
 |  - http://192.168.123.30/feed/, <generator>https://wordpress.org/?v=6.1.1<
/generator>
 |  - http://192.168.123.30/comments/feed/, <generator>https://wordpress.org/
?v=6.1.1</generator>
 |
 | [!] 20 vulnerabilities identified:
 |
 | [!] Title: WP ≤ 6.2 - Unauthenticated Blind SSRF via DNS Rebinding
 |     References:
 |      - https://wpscan.com/vulnerability/c8814e6e-78b3-4f63-a1d3-6906a84c1f
11
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2022-3590
 |      - https://blog.sonarsource.com/wordpress-core-unauthenticated-blind-s
srf/
 |
 | [!] Title: WP < 6.2.1 - Directory Traversal via Translation Files
 |     Fixed in: 6.1.2
 |     References:
 |      - https://wpscan.com/vulnerability/2999613a-b8c8-4ec0-9164-5dfe63adf6
e6
 |      - https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2023-2745
 |      - https://wordpress.org/news/2023/05/wordpress-6-2-1-maintenance-secu
rity-release/
```

Figure 15: Target WordPress version identified by WPScan

The scan identified that the target system was running WordPress version 6.1.1, which is an outdated and insecure version.

WPScan reported 20 known vulnerabilities affecting this version. These included issues such as:

- Cross-Site Scripting (XSS)
- Directory Traversal
- Content Injection
- Denial of Service
- Sensitive Data Disclosure

The presence of multiple known vulnerabilities demonstrates poor patch management and highlights the security risk of running outdated software. In real-world environments, unpatched WordPress core vulnerabilities can allow attackers to execute code, escalate privileges, or extract sensitive information.

2.2 XML-RPC Enabled – Remote Attack Surface Exposure

```
[+] XML-RPC seems to be enabled: http://192.168.123.30/xmlrpc.php
 | Found By: Link Tag (Passive Detection)
 | Confidence: 100%
 | Confirmed By: Direct Access (Aggressive Detection), 100% confidence
 | References:
 |  - http://codex.wordpress.org/XML-RPC_Pingback_API
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghos
t_scanner/
 |  - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_d
os/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlr
pc_login/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ping
back_access/
```

Figure 16: Confirmation of XML-RPC service enabled on the target system

The scan confirmed that XML-RPC is enabled on the target system.

XML-RPC provides remote procedure call functionality but is frequently abused for:

- Brute-force login attacks
- Distributed Denial of Service (DDoS) amplification
- Pingback exploitation

When enabled without restriction, XML-RPC increases the remote attack surface of the application. This configuration represents a potential entry point for attackers.

2.3 Plugin Enumeration – wp-file-manager Detected

```
[+] wp-file-manager
 | Location: http://192.168.123.30/wp-content/plugins/wp-file-manager/
 | Latest Version: 8.0.2 (up to date)
 | Last Updated: 2025-06-04T11:21:00.000Z
 | Readme: http://192.168.123.30/wp-content/plugins/wp-file-manager/readme.tx
t
 |
 | Found By: Known Locations (Aggressive Detection)
 |  - http://192.168.123.30/wp-content/plugins/wp-file-manager/, status: 200
 |
 | Version: 8.0.2 (100% confidence)
 | Found By: Readme - Stable Tag (Aggressive Detection)
 |  - http://192.168.123.30/wp-content/plugins/wp-file-manager/readme.txt
 | Confirmed By: Readme - ChangeLog Section (Aggressive Detection)
 |  - http://192.168.123.30/wp-content/plugins/wp-file-manager/readme.txt
```

Figure 17: WPScan confirms the presence of the manually installed WP File Manager plugin.

Using aggressive plugin detection, WPScan successfully identified the manually installed WP File Manager plugin (version 8.0.2).

This confirms that plugin enumeration was successful and demonstrates how additional plugins expand the attack surface of a WordPress installation.

Although the installed version was up to date, file management plugins historically present high risk because they:

- Allow file uploads and modifications
- Interact directly with server directories
- Can lead to remote code execution if vulnerable

This demonstrates the importance of monitoring third-party components.

2.4 Akismet Vulnerability Identified

```
[+] akismet
 | Location: http://192.168.123.30/wp-content/plugins/akismet/
 | Latest Version: 5.6
 | Last Updated: 2025-11-12T16:31:00.000Z
 |
 | Found By: Known Locations (Aggressive Detection)
 |  - http://192.168.123.30/wp-content/plugins/akismet/, status: 403
 |
 | [!] 1 vulnerability identified:
```

Figure 18: Aksimet plugin identified

The scan identified the Akismet plugin and reported a known historical stored Cross-Site Scripting (XSS) vulnerability affecting earlier versions.

Although the installed version was not vulnerable, WPScan successfully cross-referenced the plugin against its vulnerability database.

This confirms that:

- Plugin detection worked correctly
- Vulnerability database integration was functioning
- The scanner can identify third-party security risks

2.5 WPScan DB API OK – Verification of Database Access

This confirms that:

- The API key was valid
- The vulnerability database was successfully accessed
- Vulnerability intelligence was actively used during the scan

```
[+] WPScan DB API OK
 | Plan: free
 | Requests Done (during the scan): 5
 | Requests Remaining: 16
```

Figure 19: Verification of database access

Without the API token, detailed vulnerability information would not have been available.

2.6 Elapsed Time and Requests Done – Evidence of Aggressive Scanning

The scan reported:

- Over 117,000 requests performed
- Approximately 11 minutes elapsed time

```
[+] Finished: Tue Feb 24 15:17:17 2026
[+] Requests Done: 117240
[+] Cached Requests: 17
[+] Data Sent: 31.831 MB
[+] Data Received: 35.217 MB
[+] Memory used: 444.582 MB
[+] Elapsed time: 00:11:19
```

Figure 20: Elapsed Time and Requests

This indicates that aggressive plugin detection was executed. Aggressive scanning involves actively probing known plugin locations rather than relying solely on passive detection. While this approach increases scan duration, it improves detection accuracy and provides deeper visibility into installed components.

Step 3: Brute Forcing Login Passwords

```
┌──(kali㉿kali)-[~]
└─$ wpscan --url http://192.168.123.30 --enumerate u
_______________________________________________________________
         __          _______   _____
         \ \        / /  ___| / ____|
          \ \  /\  / / | |_  | (___   ___  __ _ _ __ ®
           > ><  x  < |  _|  \___ \ / __|/ _` | '_ \
          / /  \  \ | |    ____) | (__| (_| | | | |
         /_/    \__/ |_|   |_____/ \___|\__,_|_| |_|
```

Figure 21:: WPScan command used to enumerate WordPress usernames.

This command Enumerates WordPress usernames and identifies valid accounts that can be targeted in password attacks.

```
[+] faisa
 | Found By: Author Id Brute Forcing - Author Pattern (Aggressive Detection)
 | Confirmed By: Login Error Messages (Aggressive Detection)
```

Figure 22: WPScan enumeration of WordPress usernames revealing valid accounts.

Step 4: Scanning with API Key for Detailed Vulnerabilities

```
┌──(kali㉿kali)-[~]
└─$ wpscan --url http://192.168.123.30 \
--usernames faisa \
--passwords ~/Downloads/10000_common_passwords.txt \
--throttle 1 \
--max-threads 1 \
--force
```

Figure 23: WPScan command configured for username and password enumeration.

This command Uses discovered username (`faisa`), attempts login using a common password list. This tested weak password exposure and successfully identified the valid credential `faisa / 123456.`

```
[+] Performing password attack on Xmlrpc against 1 user/s
[SUCCESS] - faisa / 123456
Trying faisa / 123456789 Time: 00:00:00 ◇ (5 / 1000003)  0.00%  ETA: ??:??:?
?

[!] Valid Combinations Found:
 | Username: faisa, Password: 123456
```

Figure 24: WPScan identifying valid credentials

Step 5: Excluding Specific Scans and Setting a Delay

A targeted WPScan was performed to enumerate vulnerable plugins and themes using the following command:

```
┌──(kali㉿kali)-[~]
└─$ wpscan --url http://192.168.123.30 \
--enumerate vp,vt \
--throttle 1
```

Figure 25: Targeted WPScan for vulnerable plugins & themes (low-impact scan).

This scan focused exclusively on vulnerable plugins (`vp`) and vulnerable themes (`vt`), excluding unnecessary enumeration such as users or full aggressive scans. This approach reduced scan time and limited server impact.

The `--throttle 1` option introduced a one-second delay between HTTP requests, making the scan less aggressive and reducing the likelihood of detection or rate-limiting.

This demonstrates the ability to conduct a stealthier and more controlled vulnerability assessment, which is essential in real-world penetration testing scenarios.

```
[+] Finished: Tue Feb 24 16:45:15 2026
[+] Requests Done: 689
[+] Cached Requests: 9
[+] Data Sent: 184.006 KB
[+] Data Received: 410.741 KB
[+] Memory used: 292.891 MB
[+] Elapsed time: 00:00:12
```

Figure 26: Scan for vulnerable plugins and themes complete

Conclusion

I started by building a safe lab setup using two virtual machines: Kali Linux (attacker/testing machine) and a WordPress VM (target). To keep everything isolated, I configured the network using NAT and Host-Only adapters. After that, I used *ip a* to check the IP address information, then used *ping* from Kali to confirm the machines could talk to each other and the connection was working before scanning anything.

Once connectivity was confirmed, I accessed the WordPress site through the browser and used the wp-admin area to understand what was installed. From the admin dashboard I could see the site setup, including the users (such as the faisa account) and the installed plugins/themes. This helped me know what I was testing and what I should expect WPScan to find.

Next, I moved into the scanning stage using WPScan commands. I first updated WPScan's vulnerability database using my API key *(wpscan --api-token <API_KEY> --update)* so the tool had the latest vulnerability information. Then I ran scans against the WordPress URL to identify the WordPress version, confirm settings like *XML-RPC* enabled, and check for things like upload directory listing. I also used enumeration options to identify the active theme (*Twenty Twelve*) and to detect plugins and their details. During a deeper scan, WPScan successfully identified installed plugins including wp-file-manager and akismet, which showed that plugin detection and enumeration worked as intended.

After that, I followed the lab's password testing step. I downloaded the password list *(10000_common_passwords.txt)*, confirmed it existed, and then ran a brute-force test using WPScan. The tool eventually returned a successful result showing a valid login combination: faisa / 123456. This was important because it proved how quickly a weak password can be discovered when common password lists are used.

Finally, I completed a more controlled scan using selective enumeration and a delay. I used a command that focused only on vulnerable plugins and vulnerable themes *(--enumerate vp,vt)* and slowed requests using *--throttle 1*. This taught me how to run a scan in a more careful way instead of making lots of fast requests.

Overall, this lab taught me the full process from start to finish:

- Setting up the VM network safely,
- Confirming connectivity with commands,
- Checking the WordPress admin area to understand users/plugins/themes,
- Using WPScan commands to identify the WordPress version and configuration exposures,
- Enumerating plugins and themes,
- and testing password strength with a controlled brute-force attack.

Reflection

This lab provided a comprehensive, hands-on introduction to the real-world process of web application penetration testing. Working through each stage from environment configuration to active scanning and credential testing gave me a much clearer understanding of how attackers identify and exploit vulnerabilities in WordPress installations.

One of the most valuable insights was the brute-force exercise. Discovering the credential faisa / 123456 in a matter of minutes reinforced just how dangerous weak passwords are in practice, especially on administrative accounts. This is a finding that directly maps to real world breach scenarios and has changed how I think about password policy enforcement.

Regarding the effectiveness of WPScan as a tool: it performed extremely well within its intended scope. The aggressive plugin enumeration (over 117,000 requests) was thorough but also highlighted a trade-off between detection depth and stealth. The ability to throttle requests and limit enumeration to specific categories (vp, vt) was a useful demonstration of how the tool can be adapted for more realistic, less disruptive assessments.

One challenge I encountered was getting both VMs to communicate correctly across the Host-Only network, particularly ensuring the WordPress machine had the correct IP lease. Running sudo dhclient and re-checking with ip a resolved this, but it highlighted the importance of methodical troubleshooting in lab environments.

Use of Artificial Intelligence

In accordance with the University of Roehampton's AI guidelines, the following describes how AI tools were used to support the completion of this assessment.

ChatGPT was used for the following tasks:

- Reviewing and improving the overall organization and flow of the report to ensure it meets the assessment brief requirements.
- Debugging assistance during the lab environment setup, providing guidance on configuring the virtual machine network adapters and troubleshooting connectivity issues between the Kali Linux and WordPress VMs.

All technical work, screenshots, findings, and analysis represent my own independent lab work. AI was not used to generate technical findings, perform scans, or interpret screenshots.